

Generating Cohorts

Anthony G. Sena and Martijn J. Schuemie

2026-02-09

Contents

1	Guide for generating cohorts using CohortGenerator	1
1.1	Basic Example	1
1.2	Advanced Options	4

1 Guide for generating cohorts using CohortGenerator

This guide will provide examples of using the CohortGenerator package to generate cohorts in R. In this vignette, we will walk through the process of building a cohort definition set cohorts and look at the options available for generating the cohorts.

1.1 Basic Example

1.1.1 Loading a cohort definition set from ATLAS

We can create cohorts using ATLAS and download them for generation in R using the `exportCohortDefinitionSet` function of the `ROhdsiWebApi` package.

1.1.2 Loading an example cohort definition set

Here we will load a cohort definition set using some cohorts we use to test the CohortGenerator package:

```
cohortDefinitionSet <- getCohortDefinitionSet(  
  settingsFileName = "testdata/name/Cohorts.csv",  
  jsonFolder = "testdata/name/cohorts",  
  sqlFolder = "testdata/name/sql/sql_server",  
  cohortFileNameFormat = "%s",  
  cohortFileNameValue = c("cohortName"),  
  packageName = "CohortGenerator",  
  verbose = FALSE  
)  
  
cohortIds <- cohortDefinitionSet$cohortId  
cohortDefinitionSet$atlasId <- cohortDefinitionSet$cohortId  
cohortDefinitionSet$logicDescription <- ""
```

The code above constructs a `cohortDefinitionSet` data.frame using a set of cohorts that come with the CohortGenerator package. The `cohortDefinitionSet` data frame has the following columns:

```
names(cohortDefinitionSet)
```

```
#> [1] "cohortId"      "cohortName"    "json"          "sql"
#> [5] "atlasId"       "logicDescription"
```

Here is how these columns are used:

- **atlasId**: The ATLAS ID for the cohort. This provides linkage back to the ATLAS cohort definition
- **cohortId**: The cohortId will be the same as the atlasId upon export from ATLAS. We provide this column in case you'd like to alter the numbering scheme for your cohort definition set.
- **cohortName**: The name of the cohort in ATLAS.
- **sql**: The SQL used to construct the cohort.
- **json**: The Circe compliant JSON representation of the cohort definition. The CirceR package is used to create the SQL from the Circe json.
- **logicDescription**: The description of the cohort from ATLAS.

1.1.3 Saving in a study package

The `cohortDefinitionSet` contains all of the details about each cohort that we would like to use for generation. As a best practice, we recommend that you embed these cohort details into a study package. To do this, we've created a function to save the cohort definition set to the file system:

```
saveCohortDefinitionSet(  
  cohortDefinitionSet = cohortDefinitionSet,  
  settingsFileName = file.path(  
    packageRoot,  
    "inst/settings/CohortsToCreate.csv"  
  ),  
  jsonFolder = file.path(  
    packageRoot,  
    "inst/cohorts"  
  ),  
  sqlFolder = file.path(  
    packageRoot,  
    "inst/sql/sql_server"  
  )  
)
```

By default, saving the cohort definition set will create the files under a folder called `inst` which is where resources for a study package will live. Under `inst` the following folders and files are created:

- **inst/settings/CohortsToCreate.csv**: This will hold the list of cohorts found in the `cohortDefinitionSet`.
- **inst/cohorts**: This folder will contain a `.json` file per cohort definition.
- **inst/sql/sql_server**: This folder will contain a `.sql` file per cohort definition.

Your study package can later re-construct the `cohortDefinitionSet` by reading in these resources using the `getCohortDefinitionSet` function.

```

cohortDefinitionSet <- getCohortDefinitionSet(
  settingsFileName = file.path(
    packageRoot, "inst/settings/CohortsToCreate.csv"
  ),
  jsonFolder = file.path(packageRoot, "inst/cohorts"),
  sqlFolder = file.path(packageRoot, "inst/sql/sql_server")
)

```

1.1.4 Generating Cohorts

Now that we have created the `cohortDefinitionSet`, we're ready to generate our cohorts against our OMOP CDM. In this example, we will use the Eunomia data set as our CDM.

```

# Get the Eunomia connection details
connectionDetails <- Eunomia::getEunomiaConnectionDetails()

# First get the cohort table names to use for this generation task
cohortTableNames <- getCohortTableNames(cohortTable = "cg_example")

# Next create the tables on the database
createCohortTables(
  connectionDetails = connectionDetails,
  cohortTableNames = cohortTableNames,
  cohortDatabaseSchema = "main"
)

# Generate the cohort set
cohortsGenerated <- generateCohortSet(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = "main",
  cohortDatabaseSchema = "main",
  cohortTableNames = cohortTableNames,
  cohortDefinitionSet = cohortDefinitionSet
)

```

The code above starts by obtaining the `connectionDetails` from Eunomia. This is where you'll likely want to substitute your own connection information. Next, we call `getCohortTableNames` to obtain a list of `cohortTableNames` that we'll use for the generation process. We then call `createCohortTables` to create the cohort tables on the database server in the `cohortDatabaseSchema`. Once these tables are created, we use the function `generateCohortSet` to generate the `cohortDefinitionSet`. When calling `generateCohortSet`, we must specify the schema that holds our CDM (`cdmDatabaseSchema`), the location of the `cohortDatabaseSchema` and `cohortTableNames` where the cohort(s) will be generated. We will cover the other parameters available in the Advanced Options section.

If we'd like to see the results of the generation process, we can use the `getCohortCounts` method to query the cohort table for a summary of the persons and events for each cohort:

```

getCohortCounts(
  connectionDetails = connectionDetails,
  cohortDatabaseSchema = "main",
  cohortTable = cohortTableNames$cohortTable
)

```

```
#> Connecting using SQLite driver
#> Counting cohorts took 0.041 secs
```

```
#>   cohortId cohortEntries cohortSubjects
#> 1         1          1800           1800
#> 2         2           569            569
#> 3         3           266            266
#> 4         4          1750           1750
```

1.2 Advanced Options

1.2.1 Cohort Statistics (Inclusion Rule Statistics)

Cohorts defined in ATLAS may define one or more inclusion criteria as part of the cohort's logic. As part of cohort generation, we may want to capture these cohort statistics for use in other packages. For example, CohortDiagnostics has functionality that allows for review of inclusion rule statistics to understand how these rules may materialize between data sources.

Here we will review how to generate cohorts with inclusion rule statistics and how to export these results for use by downstream packages such as CohortDiagnostics. If you have constructed your cohorts in ATLAS, you can again use the `exportCohortDefinitionSet` function of the `ROhdsiWebApi` package. The `exportCohortDefinitionSet` function has an additional parameter called `generateStats` which when set to `TRUE` will include the SQL necessary to generate the cohort statistics.

Building on our previous example where we loaded a cohort set from the `CohortGenerator` package, let's include the code that will build the SQL for the cohort statistics:

```
# First construct a cohort definition set: an empty
# data frame with the cohorts to generate
cohortDefinitionSet <- CohortGenerator::createEmptyCohortDefinitionSet()

# Fill the cohort set using cohorts included in this
# package as an example
cohortJsonFiles <- list.files(path = system.file("testdata/name/cohorts", package = "CohortGenerator"),
for (i in 1:length(cohortJsonFiles)) {
  cohortJsonFileName <- cohortJsonFiles[i]
  cohortName <- tools::file_path_sans_ext(basename(cohortJsonFileName))
  # Here we read in the JSON in order to create the SQL
  # using [CirceR] (https://ohdsi.github.io/CirceR/)
  # If you have your JSON and SQL stored differently, you can
  # modify this to read your JSON/SQL files however you require
  cohortJson <- readChar(cohortJsonFileName, file.info(cohortJsonFileName)$size)
  cohortExpression <- CirceR::cohortExpressionFromJson(cohortJson)
  cohortSql <- CirceR::buildCohortQuery(cohortExpression, options = CirceR::createGenerateOptions(gener
  cohortDefinitionSet <- rbind(cohortDefinitionSet, data.frame(
    cohortId = i,
    cohortName = cohortName,
    json = cohortJson,
    sql = cohortSql,
    stringsAsFactors = FALSE
  ))
}
```

In the code above, we read in the cohort JSON files from the package and then use `CirceR` to build the cohort query SQL using the `CirceR::buildCohortQuery` command. Note that in this function we are

specifying the options = `CirceR::createGenerateOptions(generateStats = TRUE)` to indicate that the SQL should include the code necessary to compute the cohort statistics.

Next we'll create the tables to store the cohort and the cohort statistics. Then we can generate the cohorts.

```
# First get the cohort table names to use for this generation task
cohortTableNames <- getCohortTableNames(cohortTable = "stats_example")

# Next create the tables on the database
createCohortTables(
  connectionDetails = connectionDetails,
  cohortTableNames = cohortTableNames,
  cohortDatabaseSchema = "main"
)

# We can then generate the cohorts the same way as before and it will use the
# cohort statistics tables to store the results
# Generate the cohort set
generateCohortSet(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = "main",
  cohortDatabaseSchema = "main",
  cohortTableNames = cohortTableNames,
  cohortDefinitionSet = cohortDefinitionSet
)
```

At this stage, your cohorts are generated and any cohort statistics are available in the cohort statistics tables. The next step is to export the results to the file system which is done using the code below:

```
insertInclusionRuleNames(
  connectionDetails = connectionDetails,
  cohortDefinitionSet = cohortDefinitionSet,
  cohortDatabaseSchema = "main",
  cohortInclusionTable = cohortTableNames$cohortInclusionTable
)

exportCohortStatsTables(
  connectionDetails = connectionDetails,
  cohortDatabaseSchema = "main",
  cohortTableNames = cohortTableNames,
  cohortStatisticsFolder = file.path(someFolder, "InclusionStats")
)
```

The code above performs two steps. First, we insert the inclusion rule names from the Circe expressions in the `cohortDefinitionSet`. This is important since these names are not automatically inserted into the database when generating the cohorts. Second, we export the cohort statistics to the file system which will write comma separated value (CSV) files per cohort statistic table in the `InclusionStats` folder.

Once you have exported your cohort statistics, you can optionally drop the statistics tables by using the following command:

```
dropCohortStatsTables(
  connectionDetails = connectionDetails,
  cohortDatabaseSchema = "main",

```

```

    cohortTableNames = cohortTableNames
)

```

1.2.2 Incremental Mode

CohortGenerator provides an **incremental** option for some of its functions. The purpose of this **incremental** setting is to allow for the code to attempt to skip an operation if it has already completed it. For example, in the context of cohort generation we may want to keep track of cohorts that we have already generated against a source and skip it if we know the cohort definition has not changed. To illustrate incremental mode and explain how it works, we'll continue along with our example from earlier.

```

# Create a set of tables for this example
cohortTableNames <- getCohortTableNames(cohortTable = "cohort")
createCohortTables(
  connectionDetails = connectionDetails,
  cohortTableNames = cohortTableNames,
  cohortDatabaseSchema = "main",
  incremental = TRUE
)

```

As expected, the code created the cohort tables as requested. Under the hood, since **incremental = TRUE** was set, the code did a check against the database to see if the tables already exist before creating them. To verify this, we can call the function again and check the results:

```

createCohortTables(
  connectionDetails = connectionDetails,
  cohortTableNames = cohortTableNames,
  cohortDatabaseSchema = "main",
  incremental = TRUE
)

```

```

#> Connecting using SQLite driver
#> Table "cohort" already exists and in incremental mode, so not recreating it.
#> Table "cohort" already exists and in incremental mode, so not recreating it.
#> Table "cohort_inclusion" already exists and in incremental mode, so not recreating it.
#> Table "cohort_inclusion_result" already exists and in incremental mode, so not recreating it.
#> Table "cohort_inclusion_stats" already exists and in incremental mode, so not recreating it.
#> Table "cohort_summary_stats" already exists and in incremental mode, so not recreating it.
#> Table "cohort_censor_stats" already exists and in incremental mode, so not recreating it.
#> Table "cohort_checksum" already exists and in incremental mode, so not recreating it.

```

The use of **incremental = TRUE** here allows for assurance that tables and results from previous runs are preserved. Next, we can generate our **cohortDefinitionSet** in incremental mode.

```

generateCohortSet(
  connectionDetails = connectionDetails,
  cdmDatabaseSchema = "main",
  cohortDatabaseSchema = "main",
  cohortTableNames = cohortTableNames,
  cohortDefinitionSet = cohortDefinitionSet,
  incremental = TRUE
)

```

Here we indicate that we are performing this operationally incrementally by specifying `incremental = TRUE`. Once a cohort is generated in incremental mode, the cohort ID and a checksum of the cohort SQL are saved in a checksum table within the cohort database schema. If we attempt to re-generate the same cohort set in incremental mode, `generateCohortSet` will inspect the SQL for each cohort in the `cohortDefinitionSet` and if the checksum of that cohort matches the checksum found in the table for the same cohort ID, the generation is skipped. To illustrate how this looks:

```
generateCohortSet(  
  connectionDetails = connectionDetails,  
  cdmDatabaseSchema = "main",  
  cohortDatabaseSchema = "main",  
  cohortTableNames = cohortTableNames,  
  cohortDefinitionSet = cohortDefinitionSet,  
  incremental = TRUE  
)
```

```
#> Connecting using SQLite driver
```

```
#> Skipping cohorts already generated: 1, 2, 3, 4  
#> Initiating cluster consisting only of main thread  
#> Generating cohort set took 0.09 secs
```

```
options(old)  
options(rsErrorsSuppressed)
```